

基于红外干涉法的材料外延层厚度的确定与分析

摘要

本文基于双光束干涉与多光束干涉理论，建立了外延层厚度的红外干涉测量模型，运用傅里叶变换、基于网格搜索策略的最小二乘拟合完成了模型求解。

对于问题一，根据题目中描述的简化的双光束干涉，我们首先利用**几何光学原理**，建立了光线在空气、外延层和衬底界面上的传播路径。然后，我们将干涉条纹的**极值点**作为模型输入，通过分析这些极值点的波数或波长，计算出相应的干涉级数。最终我们利用**干涉方程**将外延层厚度、折射率、入射角和干涉级数等参数联系起来，并通过求解该方程得到外延层的厚度。

对于问题二，基于简化的双光束干涉模型，设计并实现了两种高效的厚度计算算法：**条纹周期分析法**和**傅里叶变换法**。研究首先对原始干涉光谱数据进行了系统的预处理，包括异常值的剔除、去除晶格吸收区域，以及采用 Savitzky-Golay 平滑滤波方法去除高频噪声，以确保数据质量。两种算法均获得了高度一致的测量结果。使用时域的条纹周期分析法，当入射角为 10° 时，我们计算得当入射角 10° 和 15° 时，外延层厚分别为 $7.733\ \mu\text{m}$ 和 $7.810\ \mu\text{m}$ ，取算术平均值得碳化硅晶片外延层的厚度为 $7.772\ \mu\text{m}$ 。；使用频域的傅里叶变换法得到，当入射角 10° 和 15° 时，外延层厚分别为 $7.864\ \mu\text{m}$ 和 $7.886\ \mu\text{m}$ ，取算术平均值得碳化硅晶片外延层的厚度为 $7.875\ \mu\text{m}$ 。条纹周期分析法和傅里叶变换法在不同入射角下的测量结果相对误差均约为 **0.99%**，均控制在 1% 以内，充分验证了算法在不同测量条件下的高可靠性和稳定性。进一步对比发现，两种方法间的测量结果相对误差也仅为 **1.31%**，表明了两种不同原理的算法在结果上的高度收敛性与可靠性。

对于问题三，针对红外干涉法中复杂的多光束干涉现象，本文从物理原理出发，系统性地推导了其发生的必要条件。研究表明，多光束干涉的显著性取决于界面的高反射率、外延层的低吸收率以及样品界面的平整度。通过定量分析，我们发现当相邻反射光束的相对振幅达到一定阈值（如 ≥ 0.1 ）时，该效应将显著影响干涉条纹的形态。随后我们在前两问的基础上，建立了基于艾里（Airy）公式的多光束干涉数学模型，并设计了相应的遍历搜索算法，通过最小二乘法优化求解，得到了硅外延层的精确厚度。并在求解完成后验算周期长度是否符合原数据。最终我们计算得到当入射角 10° 和 15° 时，外延层厚分别为 $3.367\ \mu\text{m}$ 和 $3.334\ \mu\text{m}$ ，取算术平均值得硅晶片外延层的厚度为 $3.351\ \mu\text{m}$ 。由于碳化硅材料外延层与衬底折射率差异较小，不足以产生强烈的多光束干涉效应，因此可以初步推断其影响可忽略不计。在多光束干涉条件下利用上述模型与算法的计算结果进一步证实了我们的猜测。

本研究为红外干涉法测量外延层厚度提供了系统的建模与算法框架，具有较强的理论意义和实际应用价值。

关键字： 红外干涉法 几何光学原理 傅里叶变换法 艾里（Airy）公式 网格搜索法

一、问题背景与重述

1.1 问题背景

碳化硅作为第三代半导体材料，以其优异的电学和热学性能在高功率与高频器件中得到广泛应用。外延层厚度是影响器件性能的重要参数，因此对其进行科学、准确的测量至关重要。红外干涉法能够利用外延层与衬底之间折射率差异产生的光干涉现象，实现无损厚度测量。然而，折射率随载流子浓度和光波波长变化，同时光波在层界面可能发生多次反射，产生多光束干涉，都会对测量精度造成影响。因此，建立合理的数学模型与计算算法，并分析干涉条件对结果的影响，是保证碳化硅外延层厚度测量准确性和可靠性的关键。

1.2 问题重述

根据问题背景和实验条件，本文需要建立数学模型和算法，分析研究以下问题：

问题 1 考虑最简化的测量情形，即红外光在碳化硅外延层与衬底界面仅发生一次反射和折射，从而产生双光束干涉条纹。目标是建立数学模型，以计算外延层的厚度。

问题 2 基于问题一的数学模型。设计用于外延层厚度计算的具体算法。利用在不同入射角（ 10° 和 15° ）下测得的碳化硅晶圆片干涉光谱数据，应用所设计的算法，计算该晶圆片的外延层厚度。同时，对结果的可靠性进行深入分析。

问题 3 与前两问中的简化模型不同，实际测量中光波在外延层与衬底界面可能发生多次反射和透射，形成更为复杂的多光束干涉现象。首先，我们需要从物理原理出发，推导出产生多光束干涉的必要条件，并分析这种效应如何影响我们基于双光束模型计算厚度所产生的潜在影响。其次，我们将根据推导出的必要条件，判断在不同入射角（ 10° 和 15° ）下测得的碳化硅晶圆片干涉光谱数据是否存在多光束干涉，并建立对应的数学模型和计算算法，给出厚度计算结果。最后，我们将进一步探讨多光束干涉是否也存在于碳化硅晶圆片的测试结果中。如果我们的分析表明多光束干涉确实影响了碳化硅的厚度计算精度，我们将提出并实施相应的修正方法，最终给出修正后的计算结果。

二、问题分析

2.1 问题一分析

对于问题一，该问题旨在建立一个数学模型来确定碳化硅外延层的厚度。我们的建模流程基于红外干涉法原理：首先，利用斯涅尔定律，将光的入射角与材料折射率联系起来，计算光线在外延层内的折射角。接着，我们计算从外延层表面和衬底界面反射的

两束光之间的光程差。根据光的干涉理论，这个光程差会产生特定的相位差，并在光谱中形成干涉极大值和极小值。最后，我们通过分析这些极值点的位置，来确定干涉级数，并代入到包含外延层厚度的干涉方程中，从而求解出精确的厚度值。

2.2 问题二分析

对于问题二，该问题的核心任务是基于问题一所建立的双光束干涉模型，设计并实现一套完整的算法，以计算碳化硅外延层的厚度。该问题不仅要求给出最终的计算结果，还强调了对结果可靠性的深入分析。首先，本文对于数据进行预处理，包括异常值的剔除、晶格吸收区域的筛选，以及采用 Savitzky-Golay 平滑滤波方法去除高频噪声，以确保数据质量。因此，本文采用两种不同的方法，条纹周期分析法与傅里叶变换法求解答案并且进行误差分析。同时还考虑了色散效应，建立了折射率与波长的关系以进一步提升精度。

2.3 问题三分析

问题三旨在深入探讨多光束干涉现象，这与问题一和问题二所依赖的简化双光束模型存在本质区别。该问题的核心任务可分解为以下两个部分：物理原理与必要条件推导，数据判断与建模求解。本文建立了基于艾里公式的数学模型，该模型能够全面描述光波在薄膜内的多次反射与干涉。首先我们根据反射率计算相对振幅，判断出碳化硅晶圆片的多光束干涉效应不明显，硅晶圆片的多光束干涉效应影响较大。模型求解方面，由于该模型为非线性方程，我们采用网格搜索（Grid Search）算法进行求解。该算法通过系统地遍历预设的厚度区间，计算每个厚度值所对应的理论光谱曲线，并与实验数据进行最小二乘拟合。通过寻找残差平方和（SSR）的全局最小值，并且最终检验周期是否相符，我们能够稳健地得到与实验数据最匹配的外延层厚度。

三、 模型假设

为简化问题，本文做出以下假设：

- **理想的平面和界面：**模型假设外延层和衬底的界面都是理想的平面，且相互平行。这意味着可以忽略任何宏观或微观的粗糙度，从而可以使用经典的薄膜干涉物理模型。
- **均匀的材料光学特性：**假设外延层和衬底的折射率在其各自的层内是均匀且恒定的。这允许忽略因材料缺陷、杂质聚集或晶格畸变导致的局部折射率波动。
- **光的频率不变：**模型假设光线在反射和折射过程中频率保持不变 [1]。这简化了物理模型与计算，无需考虑频率的任何偏移，使其更易于处理。

• **外延层对于光没有吸收**：SiC 外延层和 Si 外延层在 1000cm^{-1} - 4000cm^{-1} 对于光的吸收率极低，可以忽略。

四、符号说明

分类	符号	说明	单位
几何与路径参数	d	外延层厚度	μm
	d_i	第 i 个极值所对应的外延层厚度	μm
	L_{XY}	X 点到 Y 点的距离	nm
	θ_1	入射光的入射角	$^\circ$
	θ_2	入射光的折射角	$^\circ$
波长与介质参数	λ	真空波长	nm
	λ_1	选定的第一个极值处的波长	nm
	λ_i	第 i 个极值处的波长, 且 $\lambda_i > \lambda_1$	nm
	n_1	空气折射率	—
	n_2	外延层折射率	—
相位与干涉参数	δ	反射光的相位差	—
	Δ	反射光的光程差	nm
	Φ_1	A 点的相位移	—
	Φ_2	B 点的相位移	—
	P	级数	—
	P_i	第 i 个极值所对应的级数	—
	m	λ_1 和 λ_i 的级数差	—

五、问题一的模型的建立和求解

5.1 物理背景

红外干涉法是一种无损测量外延层厚度的重要方法，其基本原理是利用外延层与衬底之间折射率的差异来产生干涉现象。由于外延层与衬底通常具有不同的掺杂载流子浓度，从而表现出不同的折射率。当红外光入射到外延层时，一部分光在外延层表面反射（反射光 1），另一部分光透过外延层并在外延层/衬底界面反射（反射光 2）。这两束光因传播路径存在差异而形成干涉。

两束反射光之间的光程差决定了干涉条纹的形成位置和形态。若干涉条件满足相长干涉或相消干涉，便会在光谱中表现为一系列极值点（峰或谷）。此外，当光从光学稀

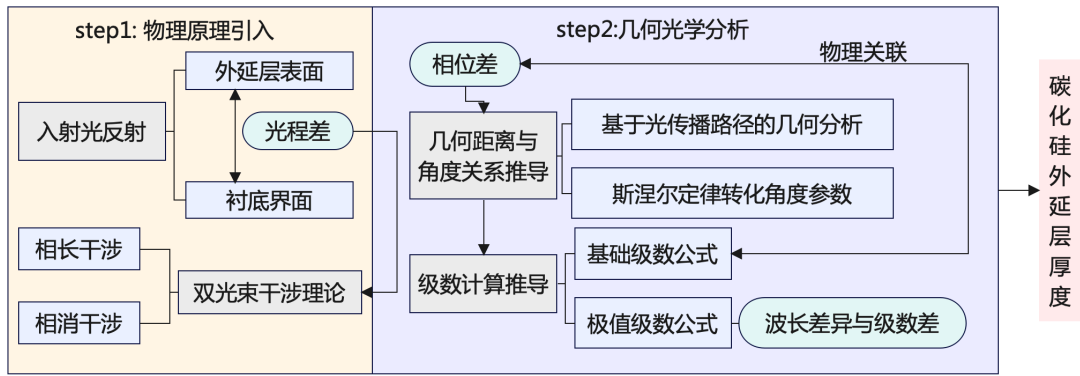


图1 问题一思路流程图

疏介质（如空气）进入光学致密介质（如外延层）时，会产生 π 的相位突变，相当于增加半个波长的光程差，这一效应通常称为“半波损失” [2]。因此，干涉条纹不仅与外延层厚度有关，还受到载流子浓度、入射角及光波长的综合影响。

通过对干涉光谱中极值点所对应的波数或波长进行测量，并结合外延层的折射率和红外光的入射角，可以建立一个数学模型来反演外延层的厚度。这一模型构成了红外干涉法测量外延层厚度的理论基础，也为问题一中建立厚度确定的数学模型提供了物理依据。

5.2 确定外延层厚度的数学模型

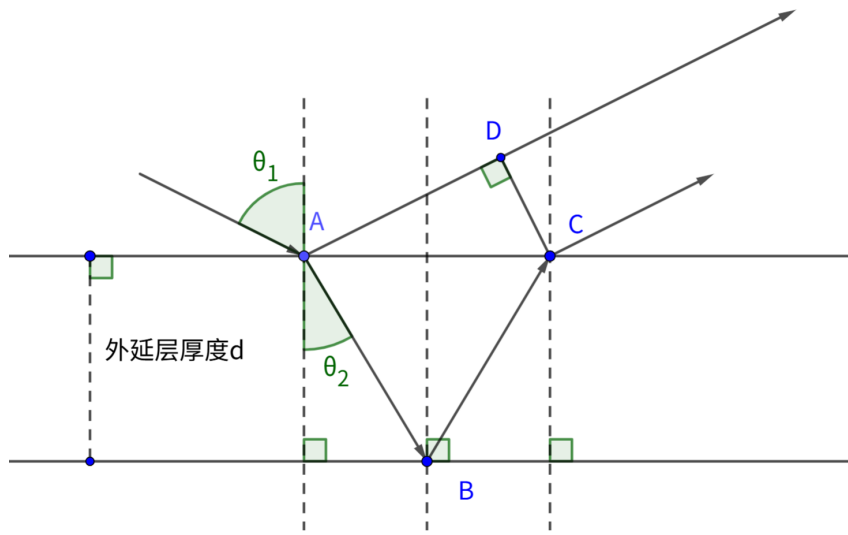


图2 碳化硅外延层的厚度检测原理示意图

如（图2）所示，入射光从A处入射，一部分光线经外延层表面AC反射（反射光1），另一部分光线经折射后在衬底界面B处反射，由C处射出（反射光2）。两束光的

相位差 δ 可根据公式 (1) 进行计算。

$$\delta = \left[\frac{2\pi(L_{AB} + L_{AC})}{\lambda} \right] n_2 - \left[\frac{2\pi L_{AD}}{\lambda} \right] + \Phi_1 - \Phi_2 \quad (1)$$

其中, δ 表示反射光的相位差, L_{AB} 、 L_{AC} 、 L_{AD} 分别表示 A 点到 B 点、C 点、D 点的距离, λ 表示真空波长, n_2 为外延层折射率, Φ_1 、 Φ_1 分别为 A 点、B 点的相位移。

根据图 2, 由几何关系可知, 外延层厚度 (d) 与 A 点到 B 点的距离 (L_{AB})、A 点到 C 点的距离 (L_{AC}) 应满足公式 (2)。A 点到 D 点的距离 (L_{AD}) 可按公式 (3) 进行计算。

$$L_{AB} + L_{AC} = \frac{2d}{\cos \theta_2} \quad (2)$$

$$L_{AD} = 2d \tan \theta_2 \sin \theta_1 \quad (3)$$

在几何光学的范畴内, 光线传播遵循特定规律。根据描述光折射现象的斯涅尔 (Snell) 定律, 入射光的入射角 θ_1 和折射角 θ_2 应满足公式 (4)。

$$\sin \theta_1 = n_2 \sin \theta_2 \quad (4)$$

为了进一步深入分析反射光的干涉特性以及外延层相关参数, 我们引入级数的概念。级数能够定量地描述反射光在干涉过程中的条纹分布等规律, 对后续研究干涉条纹极值以及外延层厚度等关键参数具有重要意义。级数 P 用于刻画反射光在干涉现象中的级数特征, 它与反射光的相位差紧密相关, 可按公式 (5) 将反射光的相位差转化为直观的级数形式, 便于后续对干涉条纹等光学现象的分析与研究。

$$P = \frac{\delta}{2\pi} \quad (5)$$

当能够观察到干涉振幅的两个极值时, 干涉条纹极值的级数反映了不同极值位置处的光学特性差异。这一级数可按公式 (6) 进行计算 [3]:

$$P_i = \frac{m\lambda_1}{\lambda_1 - \lambda_i} + 0.5 \quad (6)$$

其中, P_i 为第 i 个极值所对应的级数; m 为 λ_1 和 λ_i 的级数差; λ_1 为选定的第 1 个极值处的波长, 设为参考波长; λ_i 为第 i 个极值处的波长, 且满足 $\lambda_1 > \lambda_i$; 0.5 为光束从空气绝缘界面反射情况的常数。借助这一公式, 我们可以根据不同极值处的波长信息, 确定对应极值的级数, 为后续计算外延层厚度等参数提供依据。

第 i 个极值所对应的外延层厚度可按公式 (7) 进行计算:

$$d_i = (P_i - 0.5) \cdot \frac{0.001\lambda_i}{2\sqrt{n_2^2 - \sin^2 \theta_1}} + \frac{\Phi_1 - \Phi_2}{2\pi} \quad (7)$$

其中， d_i 为第 i 个极值所对应的外延层厚度； P_i 为第 i 个极值所对应的级数。由于相位影响主要在厚度的第三位小数以后，所以可以假设附加相位移为零，由此得到公式 (8)。

$$d_i = (P_i - 0.5) \cdot \frac{0.001\lambda_i}{2\sqrt{n_2^2 - \sin^2 \theta_1}} \quad (8)$$

六、问题二的模型的建立和求解

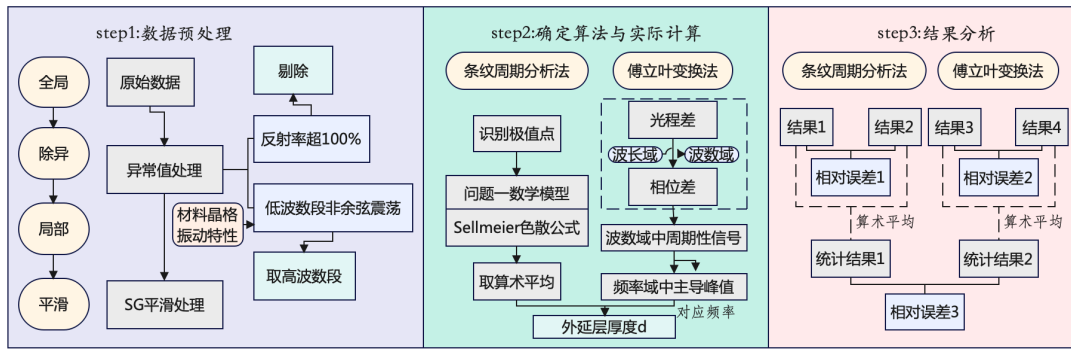


图3 问题二思路流程图

6.1 数据预处理

6.1.1 异常数据分析与剔除

在对附件 2 所提供的原始光谱数据进行分析时，我们注意到部分波数对应的反射率数值超出了 100% 这一物理极限。经研判，此类异常数据并非源于物理现象，而是实验测量过程中的多种系统性或随机性误差所致。除了可能存在的测量系统标定不准确问题，导致仪器基线或灵敏度偏移外，其他潜在原因还包括环境杂散光干扰或光源强度的不稳定，这些因素均可能在数据采集时引入额外的信号 [4]，使得检测到的反射率值虚高。

尽管这些异常值反映了数据质量的局限性，但其存在对本研究的核心结论影响甚微。本研究旨在利用干涉条纹的周期性来确定外延层厚度，其关键在于精确识别干涉光谱的峰值和谷值所对应的波数位置，而非依赖于反射率的绝对数值大小。因此，这些超出 100% 的异常数据点并未改变干涉条纹的相对位置和周期，它们仅在高反射率区域引入了数值上的偏差，而未扭曲光谱曲线的整体形态。

鉴于此，为了确保数据处理的严谨性和模型的稳健性，我们在数据预处理阶段对这些物理上不合理的异常值进行了剔除。这一步骤旨在排除数据噪音对后续分析的干扰，

使我们能够更精确地提取干涉条纹的位置信息，进而确保最终厚度计算结果的准确性和可靠性。附件一、二剔除异常数据后的波形图如图 (4) 所示。

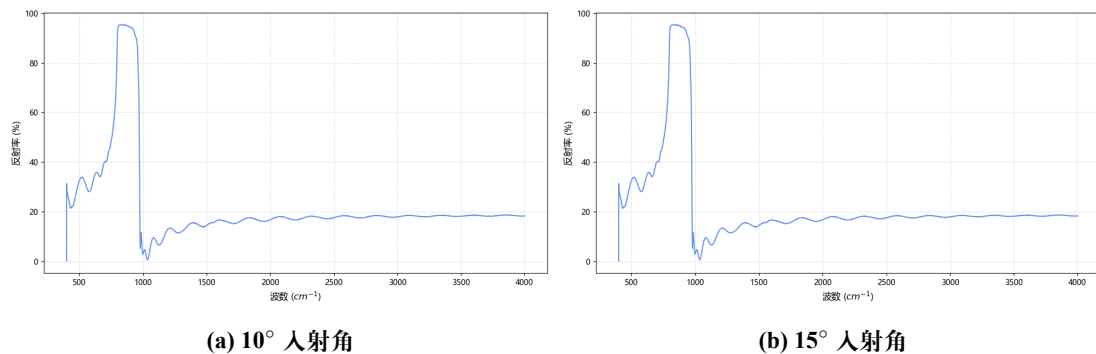


图 4 碳化硅晶圆片测试结果

在对碳化硅晶圆片外延层的红外干涉光谱进行分析时，我们观察到在低波数（通常低于约 1000 cm^{-1} ）范围内，反射率曲线不再呈现出标准干涉模型所预测的平滑余弦振荡行为，而是表现出复杂的非余弦特征，如反射率的急剧下降。这种现象并非由外延层厚度干涉所引起，而是与碳化硅材料本身的晶格振动特性 [5] 密切相关。

红外光谱法的一个重要应用是识别材料的晶格吸收峰。在特定的波数区域，入射的红外光子的能量与碳化硅晶格中的原子振动模式能量相匹配，从而发生强烈的共振吸收。对于碳化硅而言，这种强晶格共振吸收主要发生在 800 cm^{-1} 至 1000 cm^{-1} 的范围内 [6]。在该吸收区域，入射光被晶格强烈吸收，导致其反射率显著降低。这部分吸收是材料的内在物理性质，与外延层厚度所产生的干涉现象属于不同的物理机制。

因此，低波数区域光谱曲线的非余弦特征可以归因于碳化硅晶格吸收的影响。由于本研究的核心任务是利用外延层厚度所引起的干涉条纹周期性来计算厚度，而晶格吸收峰的存在不属于干涉范畴，且其强烈的吸收特性会严重扭曲并掩盖该区域内的干涉信息，故该部分数据不适用于干涉模型的计算。因此，为了保证模型的准确性和计算结果的可靠性，我们进行了数据筛选，仅选取波数在 2000 cm^{-1} 至 4000 cm^{-1} 范围内的光谱数据进行后续分析，其测试结果如图 (5) 所示。该波数范围内的光谱数据已脱离了碳化硅的晶格吸收带，其反射率曲线表现出清晰、稳定的周期性余弦振荡，这一特征完全符合外延层厚度干涉模型的理论预期。通过对这部分有效数据进行分析，我们能够准确地识别干涉峰值和谷值，进而建立可靠的数学模型来精确计算外延层的厚度。

6.1.2 Savitzky-Golay (SG) 平滑滤波

在碳化硅外延层厚度确定的研究中，我们针对附件 1 所提供的波数与反射率数据进行了预处理，以消除实验测量过程中引入的高频随机噪声，并确保后续模型的准确性。为此，我们采用了 Savitzky-Golay (SG) 平滑滤波方法。该方法基于局部多项式最小二

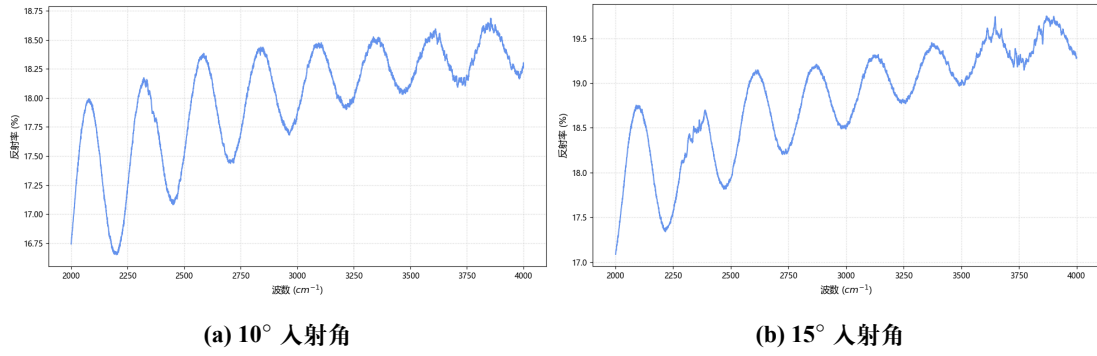


图 5 碳化硅晶圆片测试结果（筛选后）

乘拟合，在平滑数据、提升信噪比的同时，能够最大限度地保持原始光谱曲线的形态特征 [7]，这对于依赖干值和谷值位置来确定外延层厚度的模型至关重要。

我们选择的滤波器参数为：窗口大小为 51，多项式阶数为 3。具体的处理流程如下：首先，将波数与反射率数据作为时间序列输入滤波器。对于序列中的每一个数据点，滤波器均以该点为中心，截取一个包含其前后共计 50 个数据点的窗口，即窗口大小为 51。随后，利用一个 3 次多项式对窗口内的数据进行最小二乘拟合。拟合所得的多项式在窗口中心点处的值，即为该数据点平滑后的新值。这一过程对整个数据序列依次进行，最终得到一条平滑后的波数-反射率曲线，如图（6）。通过将平滑后的数据与原始数据进行对比，我们确认该处理方法有效抑制了高频振荡噪声，同时完整保留了干涉条纹的峰谷结构和整体趋势，为后续建立数学模型并求解外延层厚度提供了高质量的数据基础，从而显著提升了计算结果的可靠性与精度。

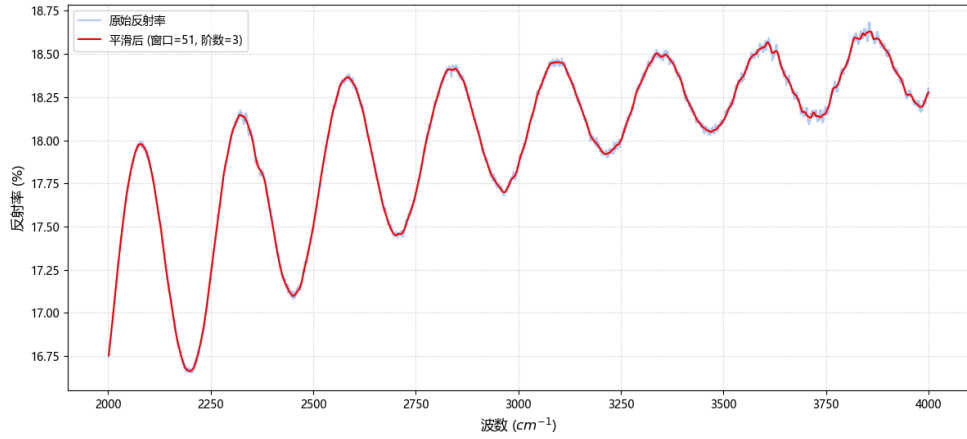
6.2 确定外延层厚度的算法

6.2.1 条纹周期分析法

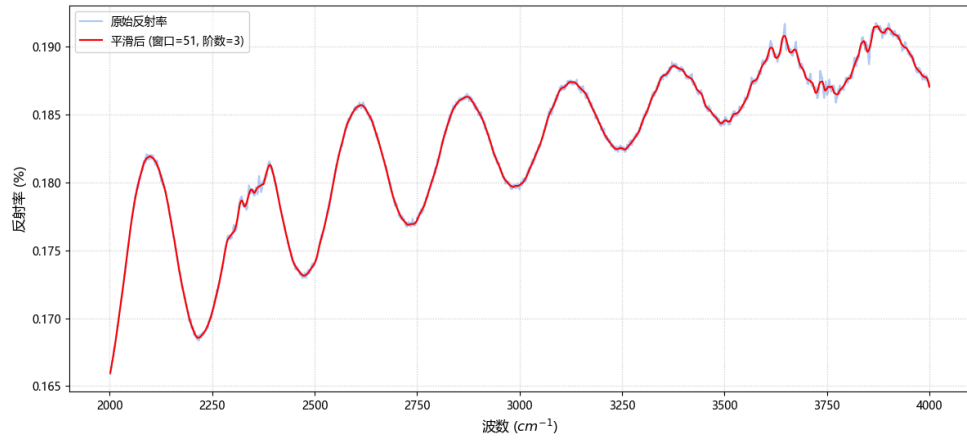
本算法旨在根据碳化硅晶圆片的光谱实测数据，确定其外延层的厚度。该方法的核心思想是，利用干涉光谱中波峰和波谷的特定位置信息，结合考虑材料色散效应的物理模型，直接计算出外延层厚度。算法通过对所有计算结果求平均，来获得最终的厚度估计值。首先，基于滑动窗口的方法识别出光的波长在 2000cm^{-1} 至 4000cm^{-1} 的所有极值点（即波峰与波谷），提取这些特征点数据以用于后续计算。选定波数最小的极值点作为参考，然后使用公式（8）进行计算。外延层的折射率 n 并非一个常数，而是随着波长 λ 变化的函数 $n(\lambda)$ ，这一现象称为色散。在本研究中，我们采用 Sellmeier 色散公式来精确描述 n 与 λ 的关系 [8]。

$$n^2(\lambda) = A + \frac{B\lambda^2}{\lambda^2 - C} \quad (9)$$

其中， A 、 B 、 C 是根据具体材料特性确定的常数。通过引入该公式，我们的模型能够更准确地反映材料的物理性质，从而提高厚度计算的精度。



(a) 10° 入射角



(b) 15° 入射角

图6 碳化硅晶圆片测试结果 (SG 平滑后)

6.2.2 傅里叶变换法

在实际工程上,更多的是使用傅里叶变换进行计算,因为它能有效滤除噪声,提高测量精度从而能更精确地找到干涉条纹的周期,进而计算出厚度。由问题一建立的数学模型,光程差 Δ 为:

$$\Delta = 2n_2d \cos \theta_2 \quad (10)$$

当我们将光谱数据从波长 (λ) 域转换为波数 ($\nu = 1/\lambda$) 域时,干涉条纹的相位差 (ϕ) 变为:

$$\phi = \frac{2\pi}{\lambda} \Delta = 2\pi(2nd \cos \theta_2)\nu \quad (11)$$

该公式表明,反射光谱在波数域中呈现出周期性,其周期频率 (f) 与厚度 (d) 存在直接的线性关系。傅里叶变换正是利用这一原理,将波数域中的周期性信号转换为频率域中的主导峰值,该峰值对应的频率即为光学厚度。

6.3 求解结果与可靠性分析

6.3.1 条纹周期分析法的求解结果

根据所建立的数学模型和相邻极值点算法，我们对附件 1 和附件 2 提供的碳化硅晶圆片的光谱实测数据进行了计算。附件 1 和附件 2 分别对应入射角为 10° 和 15° 时的测试结果。通过对干涉光谱反射率数据的处理，我们成功确定了碳化硅外延层的厚度。

具体来说，当入射角为 10° 时，我们计算得到的外延层厚 $d_{10} \approx 7.733 \mu\text{m}$ ；当入射角为 15° 时， $d_{15} \approx 7.810 \mu\text{m}$ 。这两个计算结果表明，根据我们提出的模型和算法，可以从不同入射角的光谱数据中有效地确定外延层的厚度。这些结果为进一步分析和验证模型的可靠性提供了基础

由于这两组数据是在不同条件下对同一块晶圆片进行的测量，它们的计算结果理应相同。然而，我们发现它们之间存在细微差异。为了评估我们模型计算结果的可靠性，我们通过计算这两组测量值之间的相对误差 E_r 来量化其一致性。

$$E_r = \left| \frac{d_{10} - d_{15}}{(d_{10} + d_{15})/2} \right| \times 100\% \approx \left| \frac{7.733 - 7.810}{(7.733 + 7.810)/2} \right| \times 100\% \approx 0.99\% \quad (12)$$

两次测量结果的相对误差约为 0.99%，表明其具有较高的一致性。在大多数工程与物理测量场景下，相对误差控制在 1% 以内通常可视为结果具有较好的可靠性。

6.3.2 傅里叶变换法的求解结果

根据所建立的数学模型和傅里叶变换法，我们计算得到的外延层厚 $d'_{10} \approx 7.864 \mu\text{m}$ ；当入射角为 15° 时， $d'_{15} \approx 7.886 \mu\text{m}$ 。与条纹周期分析法类似，我们计算这两组数据之间的相对误差 $E_{r'}$ ：

$$E_{r'} = \left| \frac{d'_{10} - d'_{15}}{(d'_{10} + d'_{15})/2} \right| \times 100\% \approx \left| \frac{7.864 - 7.886}{(7.864 + 7.886)/2} \right| \times 100\% \approx 0.99\% \quad (13)$$

两次测量结果的相对误差约为 0.99%，表明结果可靠性较高。

6.3.3 结果对比与分析

在完成对条纹周期分析法与傅里叶变换法的方法内可靠性验证后，为进一步评估两种测量方法的一致性及结果的整体可信度，需开展方法间的系统性对比分析。此前的方法内分析已通过计算单一方法在 10° 与 15° 入射角下测量结果的相对误差，证实了条纹周期分析法（方法内相对误差 $E_r \approx 0.99\%$ ）与傅里叶变换法（方法内相对误差 $E_{r'} \approx 0.99\%$ ）均具有良好的稳定性——两种方法对入射角的微小变化（ 5° 差异）不敏感，测量结果的波动幅度均远小于碳化硅外延层制备过程中的典型厚度偏差（ $\pm 0.1 \mu\text{m}$ ），为后续方法间对比奠定了可靠的数据基础。

鉴于两种方法的测量原理存在本质差异: 条纹周期分析法基于干涉条纹的局部极值位置差推导厚度, 依赖条纹特征的精准识别; 傅里叶变换法通过将干涉信号从空间域转换至频率域, 利用频谱峰值对应的光程差信息计算厚度, 侧重全局信号的频率特征提取, 二者的对比分析不仅能验证测量结果的收敛性, 更可通过原理互补性排除单一方法可能存在的系统性偏差, 如相邻极值点法可能因条纹重叠导致的极值误判, 或傅里叶变换法可能因噪声干扰导致的频谱峰值偏移。为实现定量对比, 首先需分别计算两种方法在不同入射角下测量结果的算术平均值 (14)(15), 以消除单一入射角测量中可能存在的偶然误差, 提升对比基准的代表性。

$$\bar{d}_1 = \frac{\sum_{i=1}^n d_i}{n} = \frac{d_{10} + d_{20}}{2} = \frac{7.733 + 7.810}{2} \approx 7.772 \quad (14)$$

$$\bar{d}_2 = \frac{\sum_{i=1}^n d_i}{n} = \frac{d_{10}' + d_{20}'}{2} = \frac{7.864 + 7.886}{2} \approx 7.875 \quad (15)$$

计算两种方法所得结果的相对误差:

$$E_{\text{方法间}} = \left| \frac{\bar{d}_1 - \bar{d}_2}{(\bar{d}_1 + \bar{d}_2)/2} \right| \times 100\% \approx \left| \frac{7.772 - 7.875}{(7.772 + 7.875)/2} \right| \times 100\% \approx 1.31\% \quad (16)$$

两种方法和两种入射角的外延层厚度测量结果如表 (1) 所示:

表 1 碳化硅外延层厚度测量结果及误差分析 (单位: μm)

测量方法	入射角		方法内相对误差
	10°	15°	
条纹周期分析法	$d_{10} = 7.733$	$d_{20} = 7.810$	$E_r \approx 0.99\%$
傅里叶变换法	$d_{10}' = 7.864$	$d_{20}' = 7.886$	$E_{r'} \approx 0.99\%$
方法间相对误差			$E_{\text{方法间}} = 1.31\%$

无论是不同方法之间, 还是同一方法在不同入射角 (10° 和 15°) 下, 相对误差都处于较小的范围, 这表明两种方法的测量结果具有良好的一致性与稳定性, 进一步说明所得到的碳化硅外延层厚度结果是可靠的。

七、问题三的模型的建立和求解

7.1 物理背景

红外干涉法的基本原理是利用外延层与衬底之间的折射率差异, 在界面处形成反射与透射光, 从而产生干涉信号。对于最简单的情形, 仅考虑外延层上表面与外延层/衬底界面之间的两束光相干叠加, 即可得到典型的双光束干涉条纹。然而, 在实际测量过

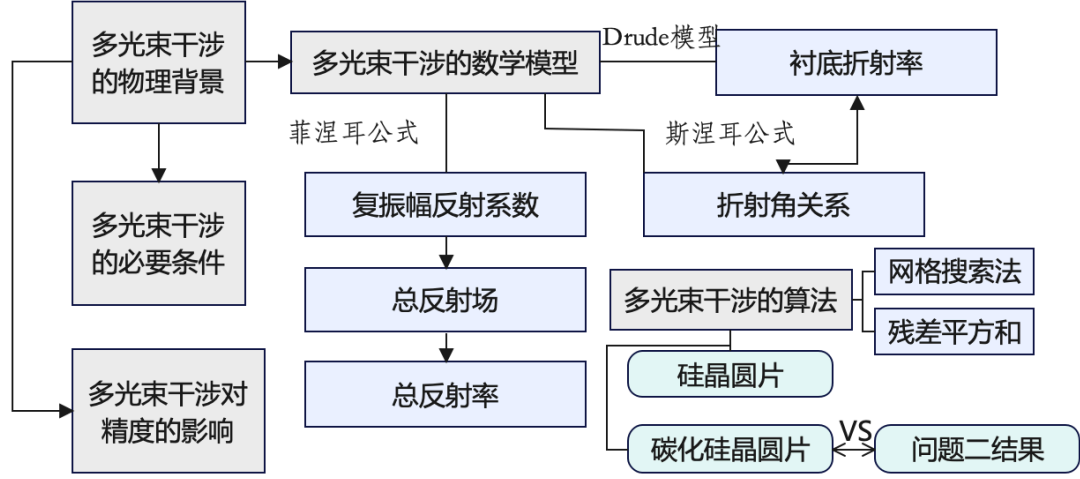


图 7 问题三思路流程图

程中，光波在外延层及衬底内部会发生多次反射与透射，使得观测到的干涉信号不再仅由两束光形成，而是呈现出多光束干涉的特征。具体而言，设入射光波长为 λ ，外延层折射率为 n ，入射角近似取为垂直入射，则光在外延层中传播的单程光程差为 nd 。每一次在外延层上下表面反射后，都会产生新的反射光分量，这些分量在光程上相互叠加。其相位差可表示为：

$$\Delta\varphi = \frac{2\pi}{\lambda} \cdot 2nd \quad (17)$$

其中因子 $2nd$ 对应于光在外延层内一次往返的光程长度。由此可见，反射信号不仅包含主干涉条纹，还受到高阶反射项的修正，呈现出典型的多光束干涉特征。在实际实验条件下，多光束干涉与双光束干涉的主要差异体现在干涉强度分布上。与两光束干涉产生的正弦波状条纹不同，多光束干涉产生的干涉条纹通常更尖锐、更密集 [9]。在实际测量中，如果外延层和衬底界面的反射率较高，或者光学系统的会聚光束条件允许，就会出现多光束干涉现象。这种情况下，基于两光束干涉模型建立的厚度计算模型可能不再准确，因为模型没有考虑多次反射所带来的额外光程和强度变化。多光束干涉的存在会使得光谱极值点的位置发生偏移，从而影响对厚度计算的精度。因此，为了获得可靠的测量结果，需要对多光束干涉进行深入分析，推导其产生的必要条件，并设计相应的数学模型和算法来消除或修正其影响。

7.2 多光束干涉的必要条件推导

根据多光束干涉的物理原理，我们先进行定性分析，再定量计算。

7.2.1 定性条件

要产生明显的多光束干涉现象，需要满足以下几个定性条件，这些条件共同确保了多次反射光束能够有效地叠加并形成稳定的干涉图样：

1. **高反射率的界面**: 干涉依赖光在介质界面的多次反射和透射。若界面反射率过低，光强经一两次反射后会快速衰减，后续反射光极弱，无法与前序光产生明显干涉。因此，界面需有足够高的反射率，才能确保多次反射效应显著。
2. **低吸收率的外延层**: 若介质对光吸收能力较强，光波多次往返外延层时，光强会大量损失，反射光变弱，多光束干涉就没明显条纹。所以外延层材料对测量用的红外光谱吸收得足够弱，才能让光多次反射。
3. **均匀且平整的样品**: 多光束干涉条纹清晰依靠反射光束光程差恒定。如果外延层厚度不均、界面不平整，光程差就会改变，干涉条纹会模糊甚至消失。
4. **高分辨率的光谱**: 多光束干涉光程差大，需光源有足够长的相干长度。光源光谱宽度过宽会使相干长度过短，不满足干涉条件，故光源光谱分辨率要足够高。

7.2.2 定量条件

多光束干涉的产生不仅需要满足上述定性条件，还需要从物理角度对相关参数进行定量分析。

1. **光程差与相干性**: 多光束干涉的核心在于光波在外延层内部经过多次反射后，相邻反射光束之间产生一个固定的光程差，从而形成稳定的干涉图样。如图（2）所示，红外光以入射角 θ_1 从空气（折射率 $n_1 \approx 1$ ）入射到外延层表面，在界面发生折射，其折射角为 θ_2 。光波在外延层内部（折射率 n_2 ）往返一次所产生的额外光程差满足公式（10）。光波能够形成干涉条纹的前提是，不同光程的光波之间具有相干性。光波的相干性由光源的相干长度 L 决定。相干长度 L 是光波能够保持稳定相位关系的传播距离，其表达式为：

$$L = \frac{\lambda^2}{\Delta\lambda} \quad (18)$$

其中， λ 是光波的中心波长， $\Delta\lambda$ 是光源的光谱线宽。为了使多光束干涉发生，要求光源的相干长度 L 必须大于或等于光波在外延层中往返一次的光程，即：

$$L \geq 2n_2d \cos \theta_2 \quad (19)$$

这一条件表明，随着外延层厚度 d 的增加，对光源相干长度的要求也随之提高，这意味着需要使用光谱线宽 $\Delta\lambda$ 更窄、分辨率更高的光源才能观察到多光束干涉现象。

2. **相对振幅与干涉显著性**: 多光束干涉的条纹清晰度，也就是干涉的显著程度，主要取决于相邻反射光束的相对振幅。当光波从空气进入外延层，再进入衬底时，会经

历多次透射和反射。

第一束反射光：

$$E_{r1} = r_{1s} E_i, \quad (20)$$

第二束反射光：

$$E_{r2} = t_{1s} \cdot r_{2s} \cdot t'_{1s} \cdot e^{i\varphi} E_i, \quad (21)$$

第三束反射光：

$$E_{r3} = t_{1s} \cdot r_{2s} \cdot r'_{1s} \cdot r_{2s} \cdot t'_{1s} \cdot e^{i2\varphi} E_i, \quad (22)$$

$\vdots$

第 m 束反射光 ($m \geq 2$):

$$E_{rm} = t_{1s} t'_{1s} r_{2s} (r'_{1s} r_{2s})^{m-2} e^{i(m-1)\varphi} E_i, \quad (23)$$

其中相位 φ 满足：

$$\varphi = \frac{4\pi n_2 d \cos \theta_2}{\lambda}, \quad (24)$$

式中： r_{1s} 为光在上界面的反射系数， t_{1s} 为光从上界面透入膜内的透射系数， r_{2s} 为光在下界面的反射系数， t'_{1s} 为光从膜内从上界面透出的透射系数， r'_{1s} 为光从膜内在上界面的反射系数。以透射光为例，相邻透射光束的电场强度之比，即振幅比 ρ ：

$$\left| \frac{E_{r(m+1)}}{E_{rm}} \right| = |r'_{1s} r_{2s} e^{i\varphi}| = |r_{1s}| \cdot |r_{2s}| \quad (25)$$

这个相对振幅 ρ 决定了干涉条纹的显著程度。如果 ρ 值很小，则后续反射光束的振幅将迅速衰减，不足以产生明显的干涉条纹。通常认为，当相对振幅 ρ 达到一定值，例如 $\rho \geq 0.1$ 时，多光束干涉效应才能显著。这需要界面具有足够高的反射率，以保证多次反射后光强仍能保持在一定水平。

7.3 多光束干涉对计算精度的影响

多光束干涉可能会对厚度计算精度产生显著影响，主要体现在以下几个方面：

1. **极值位置偏移**：当存在多光束干涉时，干涉波峰和波谷的位置会发生偏移。由于记录得到的极值波数存在误差，这会导致计算出的波长出现偏大或偏小的偏差。

2. **干涉条纹变化**: 与双光束干涉产生的正弦波形干涉条纹不同, 多光束干涉会使干涉条纹的形状变得更尖锐, 对比度更高。这使得波峰和波谷的位置更难精确确定, 从而引入更大的测量误差。
3. **模型失效**: 问题一所建立的数学模型是基于双光束干涉的假设。如果实际测量中存在多光束干涉, 该模型将不再适用, 因为它没有考虑到多重反射所引入的额外光程和相位变化。这将导致基于该模型的所有计算结果都出现系统性偏差。
4. **计算复杂性增加**: 为了准确计算厚度, 必须建立一个能够描述多光束干涉的更复杂的数学模型。这不仅增加了模型的复杂性, 也对算法的鲁棒性和计算量提出了更高的要求。未能正确处理多光束干涉将直接影响结果的可靠性和精度。

7.4 多光束干涉条件下计算外延层厚度的数学模型

基于多光束干涉理论, 本文构建了用于精准刻画光在碳化硅外延层中传播行为的数学模型。该模型突破了仅考虑外延层表面与衬底界面单次反射的局限, 全面纳入了光波在两界面间的多次反射与透射过程, 从而能更真实地反映光与外延层的相互作用。

针对平行 (p) 和垂直 (s) 偏振光, 借助菲涅尔公式计算各界面的复振幅反射系数。对于由空气 (折射率 n_1)、外延层 (折射率 n_2) 和衬底 (折射率 n_3) 构成的三层结构, 不同偏振光在界面处的复振幅反射系数可表示为方程组 (26):

$$\begin{cases} r_{1s} = \frac{n_1 \cos \theta_1 - n_2 \cos \theta_2}{n_1 \cos \theta_1 + n_2 \cos \theta_2} \\ r_{2s} = \frac{n_2 \cos \theta_2 - n_3 \cos \theta_3}{n_2 \cos \theta_2 + n_3 \cos \theta_3} \\ r_{1p} = \frac{n_2 \cos \theta_1 - n_1 \cos \theta_2}{n_2 \cos \theta_1 + n_1 \cos \theta_2} \\ r_{2p} = \frac{n_3 \cos \theta_2 - n_2 \cos \theta_3}{n_3 \cos \theta_2 + n_2 \cos \theta_3} \end{cases} \quad (26)$$

其中, θ_1 为入射角, θ_2 为外延层内的折射角, θ_3 为衬底内的折射角。

多光束干涉的总反射场是所有反射光束的叠加结果 [10]。设入射光场为 E_i , 总反射场可表示为:

$$E_r = \left[r_{1s} + t_{1s} t'_{1s} r_{2s} e^{i\varphi} \sum_{k=0}^{\infty} (r'_{1s} r_{2s} e^{i\varphi})^k \right] E_i$$

其中, r_{1s} 、 r'_{1s} 分别为界面 1 处不同方向的反射系数, t_{1s} 、 t'_{1s} 为界面 1 处不同方向的透射系数, r_{2s} 为界面 2 处的反射系数, φ 为相邻反射光束间的相位差。

由于 $|r'_{1s} r_{2s} e^{i\varphi}| < 1$, 上述级数为等比级数, 根据等比级数求和公式 $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ ($|x| < 1$), 对 $\sum_{k=0}^{\infty} (r'_{1s} r_{2s} e^{i\varphi})^k$ 求和可得:

$$\sum_{k=0}^{\infty} (r'_{1s} r_{2s} e^{i\varphi})^k = \frac{1}{1 - r'_{1s} r_{2s} e^{i\varphi}}$$

将其代入总反射场表达式，得到：

$$E_r = \left[r_{1s} + \frac{t_{1s}t'_{1s}r_{2s}e^{i\varphi}}{1 - r'_{1s}r_{2s}e^{i\varphi}} \right] E_i$$

在无吸收（能量守恒）的情况下，斯托克斯关系成立，即 $r'_{1s} = -r_{1s}$ ， $t_{1s}t'_{1s} = 1 - r_{1s}^2$ 。将其代入总反射场表达式，有：

$$E_r = \left[r_{1s} + \frac{(1 - r_{1s}^2)r_{2s}e^{i\varphi}}{1 + r_{1s}r_{2s}e^{i\varphi}} \right] E_i$$

对其进行通分合并：

$$E_r = \frac{r_{1s}(1 + r_{1s}r_{2s}e^{i\varphi}) + (1 - r_{1s}^2)r_{2s}e^{i\varphi}}{1 + r_{1s}r_{2s}e^{i\varphi}} E_i$$

化简分子部分：

$$\begin{aligned} & r_{1s}(1 + r_{1s}r_{2s}e^{i\varphi}) + (1 - r_{1s}^2)r_{2s}e^{i\varphi} \\ &= r_{1s} + r_{1s}^2r_{2s}e^{i\varphi} + r_{2s}e^{i\varphi} - r_{1s}^2r_{2s}e^{i\varphi} \\ &= r_{1s} + r_{2s}e^{i\varphi} \end{aligned}$$

因此，总反射场可简化为：

$$E_r = \frac{r_{1s} + r_{2s}e^{i\varphi}}{1 + r_{1s}r_{2s}e^{i\varphi}} E_i$$

光强反射率 R_s 定义为反射光强与入射光强的比值，即 $R_s = \left| \frac{E_r}{E_i} \right|^2$ 。令分子 $A = r_{1s} + r_{2s}e^{i\varphi}$ ，分母 $B = 1 + r_{1s}r_{2s}e^{i\varphi}$ ，则 $R_s = \frac{|A|^2}{|B|^2}$ 。

计算分子的模平方 $|A|^2$ ：

$$\begin{aligned} |A|^2 &= (r_{1s} + r_{2s} \cos \varphi + ir_{2s} \sin \varphi)(r_{1s} + r_{2s} \cos \varphi - ir_{2s} \sin \varphi) \\ &= r_{1s}^2 + r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi \end{aligned}$$

计算分母的模平方 $|B|^2$ ：

$$\begin{aligned} |B|^2 &= (1 + r_{1s}r_{2s} \cos \varphi + ir_{1s}r_{2s} \sin \varphi)(1 + r_{1s}r_{2s} \cos \varphi - ir_{1s}r_{2s} \sin \varphi) \\ &= 1 + r_{1s}^2r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi \end{aligned}$$

最终，光强反射率为：

$$R_s = \frac{r_{1s}^2 + r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi}{1 + r_{1s}^2r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi}$$

总反射率 R 是反射光束复振幅的平方和（即 $R = |r|^2$ ），它综合了所有多重反射光的干涉贡献。垂直偏振光（ s 分量）与平行偏振光（ p 分量）的总反射率可通过如下方程组表示：

$$\begin{cases} R_s = \frac{r_{1s}^2 + r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi}{1 + r_{1s}^2 r_{2s}^2 + 2r_{1s}r_{2s} \cos \varphi} \\ R_p = \frac{r_{1p}^2 + r_{2p}^2 + 2r_{1p}r_{2p} \cos \varphi}{1 + r_{1p}^2 r_{2p}^2 + 2r_{1p}r_{2p} \cos \varphi} \end{cases} \quad (27)$$

在非偏振光测试场景中，平行与垂直偏振光的贡献均等，因此总反射率 R 为二者的平均值，即：

$$R = \frac{R_s + R_p}{2} \quad (28)$$

式中，相位差 φ 是由总光程差引发的，它是影响干涉条纹的核心参数，其表达式为公式 (17)。

模型充分考虑光在各界面的折射现象，利用斯涅尔定律建立起入射角 θ_1 、外延层内折射角 θ_2 和衬底内折射角 θ_3 之间的关系：

$$n_1 \sin \theta_1 = n_2 \sin \theta_2 = n_3 \sin \theta_3 \quad (29)$$

通常取空气折射率 $n_1 = 1$ ，外延层折射率 $n_2 = 2.55$ 。对于衬底折射率 n_3 ，由于其具体数值未知，需结合实验与理论模型来确定。首先，依据 Drude 模型，衬底的折射率 n_3 可表示为本征折射率 $n_{0,3}$ 与自由载流子引起的折射率变化 $\Delta n_{fc,3}$ 之和，即：

$$n_3 = n_{0,3} + \Delta n_{fc,3}$$

其中，自由载流子导致的折射率变化 $\Delta n_{fc,3}$ 由 Drude 模型给出：

$$\Delta n_{fc,3} = -\frac{N_3 e^2 \lambda^2}{8\pi^2 c^2 \varepsilon_0 m^*}$$

式中， N_3 为衬底的自由载流子浓度， e 为电子电荷量， λ 为光波长， c 为真空中的光速， ε_0 为真空介电常数， m^* 为载流子有效质量。

然而，实际中衬底的自由载流子浓度 N_3 等参数往往难以直接获取精确值。此时，可通过实验测量衬底的反射光谱（或透射光谱），再结合前文基于多光束干涉理论建立的总反射率模型（包含 n_3 相关的反射系数），利用理论曲线与实测光谱的拟合方法反演 n_3 。具体而言，将总反射率公式中的 n_3 作为待拟合参数，调整其取值使理论计算的反射率曲线与实测光谱曲线的吻合度最高，此时对应的 n_3 即为所求的衬底折射率。

基于此模型，结合入射角度与材料参数，可计算出总反射率随波长的变化曲线。通过将该理论曲线与实测光谱数据进行对比，能够更精准地反演得到外延层的厚度。该模型通过系统整合多光束干涉、界面反射透射及折射定律等光学原理，为碳化硅外延层厚度的精确测量提供了理论支撑，有助于提升厚度表征的准确性与可靠性。

7.5 多光束干涉条件下计算外延层厚度的算法

Algorithm 1: 网格搜索求解多光束干涉模型下的外延层厚度

Data: 附件中波数与反射率数据

Result: 外延层厚度 d

```

1 第一次粗设定外延层厚度范围  $1\mu\text{m}$ - $100\mu\text{m}$ ;
2 for  $i = 1$  to  $100$  do
3   根据 Sellmeier 公式算出每个波数对应的外延层的理论折射率  $n_2$ ;
4   根据 Drude 公式和  $n_2$  计算  $n_3$ ;
5   根据 airy 公式算出理论反射率;
6   绘出理论曲线与原曲线, 并计算残差平方和;
7   if 残差平方和  $<$  当前最小残差平方和 then
8     更新最优  $d$ ;
9   end
10 end
11 运行完成后在得到的最优解附近逐步减小步长和搜索范围, 直到得到满意解;
```

7.5.1 基于网格搜索的全局优化算法

为实现碳化硅外延层厚度 d 的稳健反演, 本文采用基于网格搜索的全局优化算法。该算法通过系统性遍历预设参数空间实现全局寻优, 有效规避了局部优化算法易陷入局部极小值的缺陷, 确保在给定搜索范围内获得全局最优解, 尤其适用于薄膜干涉这类目标函数单峰特性明确的问题。

算法首先结合碳化硅外延层制备工艺特征与实验光谱干涉条纹密度分析, 设定厚度搜索区间为 $[d_{\min}, d_{\max}]$ (本研究最后一次网格搜索取值 $[1\mu\text{m}, 15\mu\text{m}]$, 覆盖实际制备可能的厚度范围及误差冗余), 并定义搜索步长 $\Delta d = 0.001 \mu\text{m}$ ——该步长远小于外延层厚度测量的精度要求 ($\pm 0.1\mu\text{m}$), 可保证不遗漏潜在最优解。搜索空间内的候选厚度值可表示为:

$$d_i = d_{\min} + i \cdot \Delta d \quad (i = 0, 1, 2, \dots, N - 1) \quad (30)$$

其中,

$$N = \left\lceil \frac{d_{\max} - d_{\min}}{\Delta d} \right\rceil$$

为候选厚度的总数量, $\lceil \cdot \rceil$ 表示向上取整运算。算法执行过程如下: 初始化最优厚度 $d_{\text{best}} = d_{\min}$ 及对应的初始评估值; 按候选厚度序列 $\{d_i\}$ 依次迭代, 对每个 d_i 调用物理模型计算对应的理论光谱, 并基于预设的评估准则对理论光谱与实验光谱的吻合度进行评估; 若当前 d_i 的评估结果优于已记录的最优值, 则更新最优厚度为 d_i 并记录当前评

估结果；遍历所有候选厚度后，输出最终的最优厚度 d_{best} 。该遍历过程穷尽搜索空间内所有可能解，从根本上避免了局部极小值陷阱，确保结果为设定范围内的全局最优解。

7.5.2 基于残差平方和的拟合优度评估机制

为实现理论模型与实验数据的定量比对，确保网格搜索算法能客观筛选出最优厚度参数，本文引入残差平方和（Sum of Squared Residuals, SSR）作为核心优化目标函数。该指标通过量化理论反射光谱与实验反射光谱的整体偏差，为厚度反演提供可计算、可比较的量化标准。残差平方和被定义为有效波数范围内，所有采样点处实测反射率与理论反射率偏差的平方累加值，其数学表达式为：

$$\text{SSR}(d) = \sum_{\nu \in \Omega} [R_{\text{meas}}(\nu) - R_{\text{model}}(d, \nu)]^2 \quad (31)$$

式中各参数的物理意义与约束条件为：- Ω 代表有效波数区间，本文基于信号质量分析选取 $\Omega = [1000, 4000] \text{ cm}^{-1}$ 。该区间既避开了低波数段（ $<1000 \text{ cm}^{-1}$ ）因晶格振动产生的非干涉噪声，又排除了高波数段（ $>4000 \text{ cm}^{-1}$ ）反射信号衰减导致的数据失真，确保参与计算的光谱信息具有物理显著性；- $R_{\text{meas}}(\nu)$ 为波数 ν 处的实验测量反射率，由傅里叶变换红外光谱仪经暗电流校正、基线扣除及 SG 平滑处理后获得，数据精度控制在 ± 0.001 范围内；- $R_{\text{model}}(d, \nu)$ 表示在给定厚度 d 下，通过艾里公式与 Sellmeier 色散模型计算的理论反射率，其计算过程严格遵循多光束干涉的相位叠加规律，确保与物理模型的自洽性。

选择 SSR 作为评估准则，主要基于其在薄膜厚度反演问题中的三个关键特性：

1. **偏差敏感性**: 通过对反射率偏差进行平方运算，SSR 能放大显著偏离点对拟合质量的影响，同时自然消除正负偏差的相互抵消，更真实地反映整体光谱的吻合程度。对于薄膜干涉这类存在周期性极值点的信号，该特性可有效区分理论曲线对实验峰值/谷值位置的捕捉精度；
2. **单峰分布特性**: 在薄膜厚度参数空间中，SSR 函数呈现严格单峰分布——当理论厚度偏离真实值时，SSR 值单调递增，且在真实厚度处取得全局最小值。这种特性与网格搜索的全局遍历策略形成完美匹配，确保算法能通过比较 SSR 值直接定位最优解；
3. **物理可解释性**: SSR 的数值大小直接对应理论模型与物理现象的偏差程度，其数量级可用于评估测量系统的精度水平。例如，对于高质量外延层样品，最优解处的 SSR 通常小于 10^{-4} ，表明理论模型能解释 99% 以上的实验信号变化。

SSR 在全局优化流程中的执行路径如下：当算法遍历至候选厚度 d_i 时，首先调用多光束干涉模型计算全波数段的理论反射率 $R_{\text{model}}(d_i, \nu)$ ；随后在有效区间 Ω 内逐点计算与实验值的偏差 $[R_{\text{meas}}(\nu) - R_{\text{model}}(d_i, \nu)]$ ，并累计平方和得到 $\text{SSR}(d_i)$ ；最后通过比较

所有候选厚度的 SSR 值，将最小 SSR 对应的 d_{best} 确定为最优厚度。这种以数据驱动的评估机制，彻底摆脱了传统厚度测量中对经验参数的依赖，使反演结果具有明确的物理意义与统计显著性。同时，SSR 的计算过程可完全复现，为不同测量系统、不同样品间的结果比对提供了统一标准。

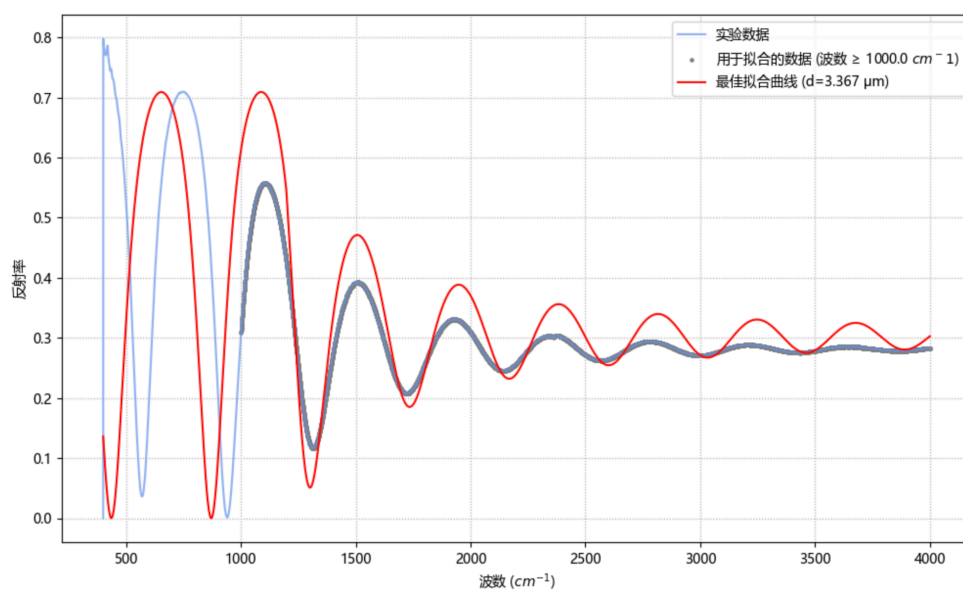
7.6 硅外延层厚度的计算结果

根据所建立的多光束干涉情况下数学模型和算法，我们对附件 3 和附件 4 提供的硅晶圆片的光谱实测数据进行了计算。附件 3 和附件 4 分别对应入射角为 10° 和 15° 时的测试结果。通过对干涉光谱反射率数据的处理，我们成功确定了硅外延层的厚度。具体来说，当入射角为 10° 时，我们计算得到的外延层厚 $d_3 \approx 3.367 \mu\text{m}$ ；当入射角为 15° 时， $d_4 \approx 3.334 \mu\text{m}$ ，拟合图如图（8）所示：

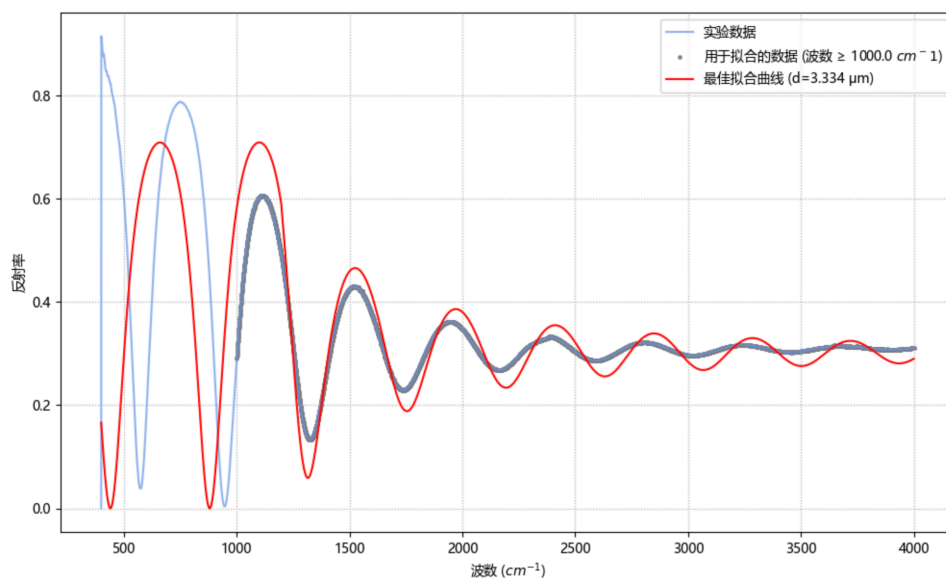
我们观察到由于实际材料对光的吸收，在多光束干涉模型中，拟合得到的理论振幅通常会大于实测光谱的振幅。这种现象是由于光在介质（例如碳化硅外延层）中传播时，会因材料自身的吸收系数而逐渐衰减，导致反射光的强度减弱，从而使得干涉条纹的振幅小于理想模型中的预测值。

7.7 多光束干涉条件下的碳化硅外延层厚度计算

首先，我们基于多光束干涉的必要条件进行推断，认为其对本研究中的计算精度影响有限。多光束干涉效应显著的前提是材料界面具有高反射率，而界面反射率与前后两种介质的折射率差异密切相关。在所研究的碳化硅晶圆片中，外延层的折射率为 2.55，衬底的折射率约为 2.7[3]，两者折射率差异较小，不足以产生强烈的多光束干涉效应，因此可以初步推断其影响可忽略不计。为了验证这一推断，我们进一步运用了基于多光束干涉理论建立的数学模型，对数据进行了重新计算。该模型考虑了光在两界面之间的多次反射和透射，能够更全面地描述干涉现象。通过求解该模型，我们得到了外延层的厚度值：当入射角为 10° 时，我们计算得到的外延层厚 $d_5 \approx 7.851 \mu\text{m}$ ；当入射角为 15° 时， $d_6 \approx 7.931 \mu\text{m}$ 。同公式（12）的方式，计算得两结果的相对误差约为 1.01%。再根据公式（16）的计算方式，计算得在多光束干涉下算得厚度与在两种双光束干涉算法下算得厚度的相对误差分别为 1.52%、0.20%。综上所述，多光束干涉条件下两组计算结果的相对误差较小，这验证了算法的可靠性；同时，多光束干涉与双光束干涉的计算结果之间同样具有较小的相对误差，进一步证实了“多光束干涉对碳化硅外延层厚度计算精度影响较小”的推测，为该厚度计算方法在实际场景中的应用提供了有力支撑。



(a) 10° 入射角



(b) 15° 入射角

图 8 基于网格搜索法的反射光谱拟合（外延层与衬底均考虑色散）

八、模型的评价

我们构建的在双光束干涉和多光束干涉背景下计算外延层厚度的模型，不仅有效且较为精准地计算出了元件的外延层厚度，同时具有以下优点：

8.1 模型的优点

- **模型综合考虑多个因素**，提出系统且较为精确的厚度计算方式。本文模型结合物理领域的基本原理、数学学科的建模方法以及算法的优化策略，充分考量了实际

物理场景中衬底折射率的波动、多光束干涉计算与双光束干涉的不同等细节，还有数学模型构建的合理性与精准性，以及算法在求解过程中的效率与稳定性等因素，

- **模型普适性与实用性较强。**此模型考虑了实际实验中较为常见的多光束干涉，也纳入了色散导致折射率变化这一因素，并且不仅仅能适用于 SiC 和 Si，能够针对不同种类的材料，调整参数进行使用，能适配多种材料相关的实验与应用场景，具有较高的应用价值。

- **双路径求解，结果可靠且模型稳定**双光束干涉模型采用条纹周期分析法与傅里叶变换法双路径求解，两种独立方法的结果高度一致，既证明了计算结果的可靠性，也充分体现了模型的良好稳定性。

8.2 模型的缺点

- 多光束干涉模型结果对实际数据的依赖性较强，需要较为精确的波数及反射率测量结果。双光束干涉模型对于数据利用率不高，只使用了极值点的数据，在样本量较小或波数较低时，能够使用的极值点数量过少，增加了数据采集和预处理的难度与成本。

- 模型的计算效率存在不足。在待优化参数数量增多或者数据规模扩大的情况下，基于网格搜索的全局优化算法会因需遍历大量网格点，导致计算量急剧上升；而基于残差平方和的拟合优度评估机制，也会随着数据量的增加，在每次评估时耗费更多时间，整体上可能影响模型在实际应用中的响应速度与效率。

参考文献及 AI 使用说明

- [1] BORN M, WOLF E. Principles of optics: Electromagnetic theory of propagation, interference and diffraction of light[M]. [S.l.]: Elsevier, 2013.
- [2] Doubao ai assistant[Z]. [S.l.]: ByteDance, 2025.
- [3] 碳化硅外延层厚度的测试红外反射法: GB/T 42905-2023[S]. 北京: 国家市场监督管理总局, 2023.
- [4] Deepseek[Z]. [S.l.]: DeepSeek Inc., 2025.
- [5] SPITZER W G, KLEINMAN D A, WALSH D. Infrared properties of hexagonal silicon carbide[J]. Physical Review, 1959, 113(1):127-132.
- [6] Gemini[Z]. [S.l.]: Google, 2025.

- [7] SCHAFER R W. What is a savitzky-golay filter?[J]. IEEE Signal Processing Magazine, 2011, 28(4):111-117.
- [8] PALIK E D. Handbook of optical constants of solids[M]. Orlando: Academic Press, 1985: 587-595.
- [9] 梁铨廷, 刘翠红. 物理光学简明教程[M]. 第 2 版. 北京: 高等教育出版社, 2001.
- [10] HECHT E. Optics[M]. Boston, MA: Pearson, 2017.

附录 A 文件列表

文件名	功能描述
geometry_measurement.py, calculate_1.py	问题二相邻极值点法程序代码
fft_sic.py	问题二傅里叶变换程序代码
reflect.py, calculator_final.py	问题三程序代码
draw_global.py, draw_local.py	画图代码

附录 B 代码

draw_global.py

```
1
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5
6
7 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
8 plt.rcParams['axes.unicode_minus'] = False
9
10
11
12
13 try:
14
15     file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件1.
16     xlsx'
17     df = pd.read_excel(file_path)
18
19     wavenumber = df['波数 (cm-1)']
20     reflectivity_original = df['反射率 (%)']
21
22 except FileNotFoundError:
23     print(f"错误：找不到数据文件 '{file_path}'!")
```

```

24
25 fig, ax = plt.subplots(figsize=(10, 6))
26
27 ax.plot(wavenumber, reflectivity_original,
28         color='cornflowerblue',
29         label='原始测量数据')
30
31 ax.set_title('样品反射率光谱分析图', fontsize=16, pad=20)
32 ax.set_xlabel('波数 ( $\text{cm}^{-1}$ )', fontsize=12)
33 ax.set_ylabel('反射率 (%)', fontsize=12)
34 ax.grid(True, linestyle=':', alpha=0.6)
35 plt.tight_layout()
36
37 plt.show()
38
39 print("绘图完成！")

```

draw_local.py

```

1
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5
6
7 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
8 plt.rcParams['axes.unicode_minus'] = False
9
10
11
12
13 try:
14
15     file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件1.
16     xlsx'
17     df = pd.read_excel(file_path)

```

```

17     wavenumber = df.iloc[3322:,0]
18     reflectivity_original = df.iloc[3322:,1]
19
20 except FileNotFoundError:
21     print(f"错误：找不到数据文件 '{file_path}'! ")
22
23 fig, ax = plt.subplots(figsize=(10, 6))
24
25 ax.plot(wavenumber, reflectivity_original,
26         color='cornflowerblue',
27         label='原始测量数据')
28
29 ax.set_title('样品反射率光谱分析图', fontsize=16, pad=20)
30 ax.set_xlabel('波数 ( $\text{cm}^{-1}$ )', fontsize=12)
31 ax.set_ylabel('反射率 (%)', fontsize=12)
32 ax.grid(True, linestyle=':', alpha=0.6)
33 plt.tight_layout()
34
35 plt.show()
36
37 print("绘图完成！")

```

geometry_measurement.py

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.signal import find_peaks
5 #找出极值点
6 def peak_and_valley(data, window_size):
7     start_index = data.index[0]
8     peaks_indices = set()
9     valleys_indices = set()
10    half_window = window_size // 2
11    data_list = data.tolist()
12    for i in range(half_window, len(data_list) - half_window):

```

```

13         if data_list[i] == max(data_list[i - half_window:i +
14                                half_window + 1]):
15             peaks_indices.add(i + start_index)
16         for i in range(half_window, len(data_list) - half_window):
17             if data_list[i] == min(data_list[i - half_window:i +
18                                    half_window + 1]):
19                 valleys_indices.add(i + start_index)
20         all_extrema = sorted(list(peaks_indices.union(
21                                valleys_indices)))
22
23         return all_extrema
24
25 #数据从2000开始读
26 file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件1.xlsx'
27 df = pd.read_excel(file_path)
28 wavenumber = df.iloc[3322:,0]
29 wavelength=df.iloc[3322:,2]
30 reflectivity_original = df.iloc[3322:,1]
31 peak_valley_location=peak_and_valley(reflectivity_original
32                                     ,400)
33 max_min_reflexity=[]
34 max_min_wavenumber=[]
35 max_min_wavelength=[]
36 for i in peak_valley_location:
37     max_min_reflexity.append(reflectivity_original[i])
38     max_min_wavenumber.append(wavenumber[i])
39     max_min_wavenumber_output=[float(x) for x in
40                                max_min_wavenumber]
41     max_min_reflexity_output=[float(x) for x in
42                                max_min_reflexity]
43     max_min_wavelength.append(wavelength[i])

```

```

42     max_min_wavelength_output=[float(x) for x in
    max_min_wavelength]
43 print(max_min_wavenumber_output)
44 print(max_min_reflexity_output)
45 print(max_min_wavelength_output)
46
47 #这里的三个数据都需要从第二个点开始，选择第二个点作为原点，每
    个波峰之间m差1，波谷与波峰差0.5
48 input("按回车键退出")

```

smooth.py

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from scipy.signal import find_peaks, savgol_filter
5  plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
6  plt.rcParams['axes.unicode_minus'] = False
7
8  #平滑曲线，sg窗口
9  file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件2.xlsx'
10 df = pd.read_excel(file_path)
11 wavenumber = df.iloc[3322:,0]
12 wavelength = df.iloc[3322:,2]
13 reflectivity_original = df.iloc[3322:,1]/103
14 window_length = 51
15 polyorder = 3
16
17 reflectivity_smoothed = savgol_filter(reflectivity_original,
18                                     window_length=
19                                     window_length,
20                                     polyorder=polyorder)
21
22 fig, ax = plt.subplots(figsize=(12, 6))
23 ax.plot(wavenumber, reflectivity_original,
24         color='cornflowerblue',

```

```

24         label='原始反射率',
25         alpha=0.5)
26
27 ax.plot(wavenumber, reflectivity_smoothed,
28         color='red',
29         linewidth=1.5,
30         label=f'平滑后 (窗口={window_length}, 阶数={polyorder
31             })')
32
33 ax.set_title('反射率数据平滑前后对比', fontsize=16, pad=20)
34 ax.set_xlabel('波数 ( $\text{cm}^{-1}$ )', fontsize=12)
35 ax.set_ylabel('反射率 (%)', fontsize=12)
36 ax.grid(True, linestyle=':', alpha=0.6)
37 ax.legend()
38 plt.tight_layout()
39 plt.show()
40
41 # 使用find_peaks函数寻找极值点
42
43 peaks, _ = find_peaks(reflectivity_smoothed, height=0,
44                       distance=100, prominence=1)
45 valleys, _ = find_peaks(-reflectivity_smoothed, height=0,
46                        distance=100, prominence=1)
47 all_extrema = np.sort(np.concatenate((peaks, valleys)))
48 max_min_reflexity = reflectivity_smoothed[all_extrema]
49 max_min_wavenumber = wavenumber.iloc[all_extrema].values
50 max_min_wavelength = wavelength.iloc[all_extrema].values
51 print("极值点波数:", max_min_wavenumber)
52 print("极值点反射率:", max_min_reflexity)
53 print("极值点波长:", max_min_wavelength)
54 fig, ax = plt.subplots(figsize=(12, 6))
55 ax.plot(wavenumber, reflectivity_smoothed,
56         color='red',
57         label='平滑后反射率')
58 ax.plot(max_min_wavenumber, max_min_reflexity,

```

```

56         'ko',
57         label='极值点')
58 for i, (x, y) in enumerate(zip(max_min_wavenumber,
    max_min_reflexity)):
59     ax.annotate(f'({x:.1f}, {y:.3f})',
60                xy=(x, y),
61                xytext=(5, 5),
62                textcoords='offset points',
63                fontsize=8,
64                bbox=dict(boxstyle="round,pad=0.3", fc="yellow
    ", alpha=0.5),
65                arrowprops=dict(arrowstyle="->",
    connectionstyle="arc3,rad=0"))
66
67 ax.set_title('平滑后反射率数据及其极值点', fontsize=16, pad
    =20)
68 ax.set_xlabel('波数 ( $\text{cm}^{-1}$ )', fontsize=12)
69 ax.set_ylabel('反射率 (%)', fontsize=12)
70 ax.grid(True, linestyle=':', alpha=0.6)
71 ax.legend()
72 plt.tight_layout()
73 plt.show()
74
75 input("按回车键退出")

```

calculator_1.py

```

1 from geometry_measurement import max_min_wavenumber_output,
    max_min_reflexity_output,max_min_wavelength_output
2 import math
3 from math import sin
4
5 theta1=10
6 theta1 = math.radians(theta1)
7 #这三个数据都需要从1开始使用，0不行
8 mylambda=max_min_wavelength_output

```

```

9 valley= mylambda[2::2]
10 peak = mylambda[1::2]
11
12
13
14 #考虑n是函数，用每个极值计算都可以得到一个d，取平均
15 #要带μm单位的波长，但是本来是nm的
16 peak = [x / 1000 for x in peak]
17 A,B=4.23,2.32
18 C=0.00339
19 a=[2/i*math.sqrt(A+B*i**2/(i**2-C)-sin(theta1)**2) for i in
    peak]
20 d_peak=[1/(a[i+1]-a[i]) for i in range(len(a)-1)]
21 print(d_peak)
22 valley=[x/1000 for x in valley]
23 b=[2/i*math.sqrt(A+B*i**2/(i**2-C)-sin(theta1)**2) for i in
    valley]
24 d_valley=[1/(b[i+1]-b[i]) for i in range(len(b)-1)]
25 print(d_valley)
26 d_sum=0
27 for i in d_peak:
28     d_sum+=i
29 for i in d_valley:
30     d_sum+=i
31 d=d_sum/(len(d_peak)+len(d_valley))
32 print(d)

```

fft_sic.py

```

1 import numpy as np
2 import pandas as pd
3 from scipy import signal
4 from scipy.fft import fft, fftfreq
5
6 class EpitaxialThickness:
7     def __init__(self, wavenumber, reflectance, incident_angle

```

```

, n=2.55, start_row=3322):
8     """
9     参数:
10    - wavenumber: 波数 (cm-1)
11    - reflectance: 反射率 (%)
12    - incident_angle: 入射角 (度)
13    - n: 外延层折射率
14    - start_row: 周期性数据开始的行数
15    """
16    self.k_full = np.array(wavenumber)
17    self.R_full = np.array(reflectance)
18    self.theta = np.radians(incident_angle)
19    self.n = n
20
21    # 计算折射角
22    sin_theta2 = np.sin(self.theta) / self.n
23    self.theta2 = np.arcsin(sin_theta2)
24    self.cos_theta2 = np.cos(self.theta2)
25
26    # 提取周期性数据
27    if start_row < len(self.k_full):
28        self.k = self.k_full[start_row:]
29        self.R = self.R_full[start_row:]
30    else:
31        self.k = self.k_full
32        self.R = self.R_full
33
34    def calculate_thickness(self):
35        """计算厚度"""
36        results = []
37
38
39
40    # 方法2: FFT法
41    R_detrend = signal.detrend(self.R)

```

```

42     N = 4096
43     k_uniform = np.linspace(self.k.min(), self.k.max(), N)
44     R_interp = np.interp(k_uniform, self.k, R_detrend)
45
46     fft_result = fft(R_interp)
47     dk = k_uniform[1] - k_uniform[0]
48     freq = fftfreq(N, d=dk)
49
50     pos_mask = freq > 0
51     freq_pos = freq[pos_mask]
52     mag_pos = np.abs(fft_result[pos_mask])
53
54     peak_idx = np.argmax(mag_pos)
55     f_dominant = freq_pos[peak_idx]
56
57     d_fft = f_dominant / (2 * self.n * self.cos_theta2) #
cm
58     d_fft_um = d_fft * 1e4 # μm
59     results.append(d_fft_um)
60     print(f"FFT法: {d_fft_um:.3f} μm")
61
62 if __name__ == "__main__":
63
64
65     data = pd.read_excel(r'C:\Users\Think\Desktop\国赛\B题\附件\附件1.xlsx')
66     wavenumber = data.iloc[:, 0].values
67     reflectance = data.iloc[:, 1].values
68     calc = EpitaxialThickness(wavenumber, reflectance,
69                               incident_angle=10)
70     thickness = calc.calculate_thickness()

```

reflect.py

```
1 import math
```

```

2
3 #计算反射率判断是否存在多光束干涉
4 def calculate_reflectance(n1, n2, theta1_deg):
5
6
7     theta1 = math.radians(theta1_deg)
8
9
10    sin_theta2 = (n1 / n2) * math.sin(theta1)
11    theta2 = math.asin(sin_theta2)
12
13
14    numerator_s = n1 * math.cos(theta1) - n2 * math.cos(theta2
15    )
16    denominator_s = n1 * math.cos(theta1) + n2 * math.cos(
17    theta2)
18    rs = numerator_s / denominator_s
19    Rs = abs(rs)**2
20
21    numerator_p = n2 * math.cos(theta1) - n1 * math.cos(theta2
22    )
23    denominator_p = n2 * math.cos(theta1) + n1 * math.cos(
24    theta2)
25    rp = numerator_p / denominator_p
26    Rp = abs(rp)**2
27
28    R = (Rs + Rp) / 2
29
30    return R
31
32 print(calculate_reflectance(n1=2.55, n2=2.485, theta1_deg=15))
33 print(calculate_reflectance(n1=3.41, n2=3.26, theta1_deg=15))

```

calculate_final.py

```

1 import pandas as pd

```

```

2 import numpy as np
3 import matplotlib.pyplot as plt
4 from tqdm import tqdm
5 from scipy.optimize import curve_fit
6 import scipy.constants as const
7 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
8 plt.rcParams['axes.unicode_minus'] = False
9 #多光束干涉计算附件3,4
10 e = const.e
11 m_e = const.m_e
12 epsilon_0 = const.epsilon_0
13 c = const.c
14 N_sub = 5e19 * 1e6
15 m_star = 0.26 * m_e
16 mu = 85e-4
17 omega_p = np.sqrt((N_sub * e**2) / (epsilon_0 * m_star))
18 gamma = e / (mu * m_star)
19 epsilon_inf = 11.7
20 def n_sub_drude(wavelength_um):
21     lambda_m = wavelength_um * 1e-6
22     omega = 2 * np.pi * c / lambda_m
23     epsilon_real = epsilon_inf - (omega_p**2 * omega**2) / (
24         omega**2 * (omega**2 + gamma**2))
25     return np.sqrt(np.maximum(epsilon_real, 1.0))
26 def n_Si_model(wavelength_um):
27     #适用于轻掺杂外延层
28
29     B1 = 10.6684293
30     C1 = 0.0909121907
31
32     n_squared = 1 + (B1 * wavelength_um**2) / (wavelength_um
33         **2 - C1)
34     return np.sqrt(np.abs(n_squared))
35 def reflectance_model(wavenumber_cm, d_um):

```

```

35     """计算理论反射率（包含外延层和衬底的色散效应）"""
36     if np.isscalar(wavenumber_cm):
37         wavenumber_cm = np.array([wavenumber_cm])
38         return_scalar = True
39     else:
40         return_scalar = False
41
42     n1 = 1.0
43
44     wavelength_um = 10000 / wavenumber_cm
45
46     n2 = n_Si_model(wavelength_um)
47     n3 = n_sub_drude(wavelength_um)
48     theta1_deg = 15.0
49     theta1_rad = np.deg2rad(theta1_deg)
50     d_cm = d_um * 1e-4 # 厚度转换 (μm to cm)
51
52     sin_theta2 = n1 * np.sin(theta1_rad) / n2
53     theta2_rad = np.arcsin(sin_theta2)
54
55     sin_theta3 = n2 * np.sin(theta2_rad) / n3
56     theta3_rad = np.arcsin(sin_theta3)
57
58     # s-和p-极化反射系数
59     r1s = (n1 * np.cos(theta1_rad) - n2 * np.cos(theta2_rad))
60     / (n1 * np.cos(theta1_rad) + n2 * np.cos(theta2_rad))
61     r2s = (n2 * np.cos(theta2_rad) - n3 * np.cos(theta3_rad))
62     / (n2 * np.cos(theta2_rad) + n3 * np.cos(theta3_rad))
63     r1p = (n2 * np.cos(theta1_rad) - n1 * np.cos(theta2_rad))
64     / (n2 * np.cos(theta1_rad) + n1 * np.cos(theta2_rad))
65     r2p = (n3 * np.cos(theta2_rad) - n2 * np.cos(theta3_rad))
66     / (n3 * np.cos(theta2_rad) + n2 * np.cos(theta3_rad))
67
68     # 相移
69     phi = 4 * np.pi * n2 * d_cm * np.cos(theta2_rad) *
70     wavenumber_cm

```

```

65     # Airy
66     Rs = (r1s**2 + r2s**2 + 2 * r1s * r2s * np.cos(phi)) / (1
+ r1s**2 * r2s**2 + 2 * r1s * r2s * np.cos(phi))
67     Rp = (r1p**2 + r2p**2 + 2 * r1p * r2p * np.cos(phi)) / (1
+ r1p**2 * r2p**2 + 2 * r1p * r2p * np.cos(phi))
68
69     result = (Rs + Rp) / 2
70
71     if return_scalar:
72         return result[0]
73     else:
74         return result
75 file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件4.xlsx'
76 df = pd.read_excel(file_path, header=None, names=['wavenumber'
, 'reflectance', 'wavelength'])
77 df['wavenumber'] = pd.to_numeric(df['wavenumber'], errors='
coerce')
78 df['reflectance'] = pd.to_numeric(df['reflectance'], errors='
coerce')
79 df.dropna(inplace=True)
80 df['reflectance'] = df['reflectance'] / 100.0
81 fit_start_wavenumber = 1000.0
82 df_fit = df[df['wavenumber'] >= fit_start_wavenumber].copy()
83 x_data = df_fit['wavenumber'].values
84 y_data = df_fit['reflectance'].values
85 d_min = 1.0
86 d_max = 15.0
87 d_step = 0.001
88 d_range = np.arange(d_min, d_max + d_step, d_step)
89 errors = []
90 for d_candidate in tqdm(d_range):
91     y_theory = reflectance_model(x_data, d_candidate)
92     ssr = np.sum((y_data - y_theory)**2)
93     errors.append(ssr)
94 min_error = min(errors)

```

```

95 best_d = d_range[errors.index(min_error)]
96 #输出与画图
97 print("-" * 30)
98 print(f"网格搜索完成！")
99 print(f"计算出的最优外延层厚度 (d): {best_d:.4f} μm")
100 print(f"最小残差平方和 (SSR): {min_error:.6f}")
101 plt.figure(figsize=(12, 7))
102 plt.plot(df['wavenumber'], df['reflectance'], label='实验数据',
          , color='cornflowerblue', alpha=0.7)
103 plt.scatter(x_data, y_data, label=f'用于拟合的数据 (波数 ≥ {
          fit_start_wavenumber} $cm^{-1}$)', s=5, color='gray')
104
105 full_wavenumber_np = df['wavenumber'].values
106 y_theory_full = reflectance_model(full_wavenumber_np, best_d)
107
108 plt.plot(full_wavenumber_np, y_theory_full, 'r-', label=f'最佳
          拟合曲线 (d={best_d:.3f} μm)')
109
110 plt.xlabel('波数 ($cm^{-1}$)')
111 plt.ylabel('反射率')
112 plt.legend()
113 plt.grid(True, linestyle=':')
114 plt.show()
115
116
117 d_range_plot = np.linspace(best_d - 1.0, best_d + 1.0, 200)
118 errors_plot = []
119 for d_candidate in tqdm(d_range_plot):
120     y_theory = reflectance_model(x_data, d_candidate)
121     ssr = np.sum((y_data - y_theory)**2)
122     errors_plot.append(ssr)
123
124 plt.figure(figsize=(12, 7))
125 plt.plot(d_range_plot, errors_plot)
126 plt.scatter([best_d], [min_error], color='red', s=100, zorder

```

```

    =5, label=f'全局最小值点 d={best_d:.3f} μm')
127 plt.title('残差平方和 (SSR) 与厚度 (d) 的关系')
128 plt.xlabel('厚度 d (μm)')
129 plt.ylabel('残差平方和 (SSR)')
130 plt.legend()
131 plt.grid(True, linestyle=':')
132 plt.show()
133
134 y_theory_fit = reflectance_model(x_data, best_d)
135 residuals = y_data - y_theory_fit
136
137 plt.figure(figsize=(12, 7))
138 plt.scatter(x_data, residuals, s=10, alpha=0.6, label='残差 (
    实验值 - 拟合值)')
139 plt.axhline(y=0, color='r', linestyle='--', label='y = 0 (理想
    零误差线)')
140 plt.title('残差分析图 (考虑外延层和衬底的色散效应)')
141 plt.xlabel('波数 (cm-1)')
142 plt.ylabel('残差 (反射率误差)')
143 plt.legend()
144 plt.grid(True, linestyle=':')
145 plt.show()

```

calculate_3_12.py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from tqdm import tqdm
5 #这里是用多光束的模型算附件1,2检验
6 def n_SiC_6H_Choyke(wavelength_um):
7     """
8     根据Sellmeier公式计算SiC的折射率
9     参数: wavelength_um : 波长, 单位是微米(μm)
10    返回: 折射率 n
11    """

```

```

12     A = 4.23
13     B = 2.32
14     C_squared = 0.0339
15
16     n_squared = A + (B * wavelength_um**2) / (wavelength_um**2
17         - C_squared)
18     return np.sqrt(np.abs(n_squared))
19 #算反射率
20 def reflectance_model(wavenumber_cm, d_um):
21     if np.isscalar(wavenumber_cm):
22         wavenumber_cm = np.array([wavenumber_cm])
23         return_scalar = True
24     else:
25         return_scalar = False
26
27     n1 = 1.0
28     n3 = 2.38
29     wavelength_um = 10000 / wavenumber_cm
30     n2 = n_SiC_6H_Choyke(wavelength_um)
31     theta1_deg = 10.0
32     theta1_rad = np.deg2rad(theta1_deg)
33     d_cm = d_um * 1e-4
34     sin_theta2 = n1 * np.sin(theta1_rad) / n2
35     theta2_rad = np.arcsin(sin_theta2)
36     sin_theta3 = n2 * np.sin(theta2_rad) / n3
37     theta3_rad = np.arcsin(sin_theta3)
38     r1s = (n1 * np.cos(theta1_rad) - n2 * np.cos(theta2_rad))
39     / (n1 * np.cos(theta1_rad) + n2 * np.cos(theta2_rad))
40     r2s = (n2 * np.cos(theta2_rad) - n3 * np.cos(theta3_rad))
41     / (n2 * np.cos(theta2_rad) + n3 * np.cos(theta3_rad))
42     r1p = (n2 * np.cos(theta1_rad) - n1 * np.cos(theta2_rad))
43     / (n2 * np.cos(theta1_rad) + n1 * np.cos(theta2_rad))
44     r2p = (n3 * np.cos(theta2_rad) - n2 * np.cos(theta3_rad))
45     / (n3 * np.cos(theta2_rad) + n2 * np.cos(theta3_rad))

```

```

42     phi = 4 * np.pi * n2 * d_cm * np.cos(theta2_rad) *
wavenumber_cm
43     Rs = (r1s**2 + r2s**2 + 2 * r1s * r2s * np.cos(phi)) / (1
+ r1s**2 * r2s**2 + 2 * r1s * r2s * np.cos(phi))
44     Rp = (r1p**2 + r2p**2 + 2 * r1p * r2p * np.cos(phi)) / (1
+ r1p**2 * r2p**2 + 2 * r1p * r2p * np.cos(phi))
45     result = (Rs + Rp) / 2
46     if return_scalar:
47         return result[0]
48     else:
49         return result
50 file_path = r'C:\Users\Think\Desktop\国赛\B题\附件\附件1.xlsx'
51 df = pd.read_excel(file_path, header=None, names=['wavenumber'
, 'reflectance', 'wavelength'])
52 df['wavenumber'] = pd.to_numeric(df['wavenumber'], errors='
coerce')
53 df['reflectance'] = pd.to_numeric(df['reflectance'], errors='
coerce')
54 df.dropna(inplace=True)
55 df['reflectance'] = df['reflectance'] / 100.0
56 fit_start_wavenumber = 2000.0
57 df_fit = df[df['wavenumber'] >= fit_start_wavenumber].copy()
58 x_data = df_fit['wavenumber'].values
59 y_data = df_fit['reflectance'].values
60
61
62 d_min = 5.0
63 d_max = 12.0
64 step = 0.001
65 d_range = np.arange(d_min, d_max, step)
66 errors = [] # 存储每个d值对应的误差
67 min_error = float('inf')
68 best_d = None
69 print("开始遍历搜索最优厚度 d ...")
70 for d_candidate in tqdm(d_range):

```

```

71     y_theory = reflectance_model(x_data, d_candidate)
72     ssr = np.sum((y_data - y_theory)**2)
73     errors.append(ssr)
74     if ssr < min_error:
75         min_error = ssr
76         best_d = d_candidate
77 print("-" * 30)
78 print(f"遍历搜索完成！")
79 print(f"搜索范围: {d_min} to {d_max} μm, 步长: {step} μm")
80 print(f"计算出的最优外延层厚度 (d): {best_d:.4f} μm")
81 print(f"最小残差平方和 (SSR): {min_error:.6f}")
82 plt.figure(figsize=(12, 7))
83 plt.plot(df['wavenumber'], df['reflectance'], label='Full
    Experimental Data', color='cornflowerblue', alpha=0.7)
84 plt.scatter(x_data, y_data, label=f'Data Used for Fitting (v
    >= {fit_start_wavenumber})', s=5, color='gray')
85 full_wavenumber_np = df['wavenumber'].values
86 y_theory_full = np.zeros_like(full_wavenumber_np)
87 for i, w in enumerate(full_wavenumber_np):
88     y_theory_full[i] = reflectance_model(w, best_d)
89
90 plt.plot(full_wavenumber_np, y_theory_full, 'r-', label=f'Best
    Fit Curve (d={best_d:.3f} μm)')
91 plt.title('Reflectance Spectrum Fitting with Dispersion (Grid
    Search Method)')
92 plt.xlabel('Wavenumber (cm-1)')
93 plt.ylabel('Reflectance')
94 plt.legend()
95 plt.grid(True, linestyle=':')
96 plt.show()
97
98 # 2. 绘制误差(SSR)随d值变化的曲线
99 plt.figure(figsize=(12, 7))
100 plt.plot(d_range, errors)
101 plt.scatter([best_d], [min_error], color='red', s=100, zorder

```

```

    =5, label=f'Global Minimum at d={best_d:.3f} μm')
102 plt.title('Sum of Squared Residuals (SSR) vs. Thickness (d)')
103 plt.xlabel('Thickness d (μm)')
104 plt.ylabel('SSR (Fitting Error)')
105 plt.legend()
106 plt.grid(True, linestyle=':')
107 plt.show()
108 y_theory_fit = reflectance_model(x_data, best_d)
109 residuals = y_data - y_theory_fit
110 rmse = np.sqrt(min_error / len(y_data))
111 mae = np.mean(np.abs(residuals))
112 print(f"均方根误差 (RMSE): {rmse:.6f}")
113 print(f"平均绝对误差 (MAE): {mae:.6f}")

```